10. A* Algorithm

Aim:

To implement the A* search algorithm to find the shortest path from a start node to a goal node.

Algorithm:

1. Initialize the start node with cost zero.
2. Calculate \(f(n)=g(n)+h(n)\), where \(g(n)\) is the path cost and \(h(n)\) is the estimated cost to the goal.
3. Select the node with the lowest \(f(n)\).
4. Expand its unvisited neighbors and update their costs.
5. Repeat until the goal node is reached.
6. Display the shortest path and total cost.

Program:

```
import heapq
graph = {
'A': [('B', 1), ('C', 4)],
'B': [('D', 2), ('E', 5)],
'C': [('F', 3)],
'D': [('G', 4)],
'E': [('G', 2)],
'F': [('G', 2)],
'G': []
}
h = {
'A': 5, 'B': 4, 'C': 3,
'D': 4, 'E': 2, 'F': 2, 'G': 0
}
def aster (start, goal):
pq = [(h[start], 0, start, [start])]
```

```
visited = set ()
while pq:
f, g, node, path = heapq. heap pop(pq)
if node == goal:
print ("Path:", path)
print ("Cost:", g)
return
if node in visited:
continue
visited.add(node)
for neighbour, cost in graph[node]:
if neighbour not in visited:
new_g = g + cost
new = new_g + h[neighbour]
heapq.heappush(
pq, (new, new_g, neighbour,
path + [neighbour])
)
Aster ('A', 'G')
```

Output:

```
============================== RESTART: D:/AI/d
Best tour: (0, 1, 3, 2, 0)
Minimum cost: 80
```

Result:

Thus, the A* search algorithm was implemented successfully to find a minimum-cost path.